Filtering and projection using the Helmholtz Equation and a smoothed Heaviside step function

In this exercise you will solve the Helmholtz equation with zero Neumann conditions,

$$-R^2\nabla^2\tilde{\rho} + \tilde{\rho} = \rho \ \text{ in } \Omega \quad \text{with} \quad \hat{n}\cdot\nabla\tilde{\rho} = 0 \ \text{ on } \partial\Omega,$$

for the unknown function $\tilde{\rho}$. The equation includes a parameter *R* and an input function ρ.

We will also investigate a smoothed Heaviside function

$$H(\beta,\eta,x) = \frac{\tanh(\beta\eta) + \tanh\left(\beta(x-\eta)\right)}{\tanh(\beta\eta) + \tanh\left(\beta(1-\eta)\right)}$$

Parts of the solution can be used later in your Topology Optimization implementations.

1. Rewrite the equation to the weak form, which you can implement in FEniCS.

2. Define a 2D mesh of a square with side length 10 centered in (0,0). Use

   mesh = RectangleMesh.create([Point(-5, -5), Point(5,5)],[nx,ny],CellType.Type.triangle)

   nx and ny are the number of elements in x and y directions.
   (CellType.Type.quadrilateral is also possible)

3. Define an input function to be a disc, $\rho(x,y) = \begin{cases} 1 \text{ if } x^2+y^2 \le 3^2 \\ 0 \text{ otherwise} \end{cases}$.
   This can be defined in fenics as

```
class MyExpression(UserExpression):
    def eval(self, value, x):
        if (x[0]*x[0]+x[1]*x[1])<=3*3:
            value[0]=1.0
        else:
            value[0]=0.0
    def value_shape(self):
        return (1,)
```

   Use either ‘interpolate’ or ‘project’ to turn this into a function on an appropriate function space. The function space could, e.g., be the space of piecewise linear functions

   Vc=FunctionSpace(mesh,'Lagrange',1)

   or the space of piecewise constant functions

   Vd=FunctionSpace(mesh,'DG',0)

   This is then used to create the function, for example, as

```
rho=interpolate( MyExpression(element=Vd.ufl_element()) ,Vd)
```

A piecewise constant function space is appropriate for defining material parameters, which often are constant in large regions and change abruptly at material boundaries.

4. Choose an appropriate function space for $\tilde{\rho}$ and solve the problem using

   ```
   solve(a == L,rho_tilde)
   ```

   to obtain $\tilde{\rho}$ for R=0.1. Calculate the minimum and maximum values of $\tilde{\rho}$ and ρ. Observe how these change as you decrease the mesh size. Plot also $\tilde{\rho}$ and ρ using e.g.

   ```
   import matplotlib.pyplot as plt
   plt.figure(1)
   p=plot(rho)
   plt.title(r'$\rho$')
   plt.colorbar(p)
   plt.figure(2)
   p=plot(rho_tilde, mode='color')
   plt.title(r'$\tilde{\rho}$')
   plt.colorbar(p)
   plt.show()
   ```

   Look also at what happens (for some fixed mesh size) as you change the *R* parameter.

5. Solver parameters can be changed using extra parameters to the solve function. The installed solvers can be seen with the lines

   ```
   list_linear_solver_methods()
   list_krylov_solver_preconditioners()
   ```

   Try to change the solver using e.g.

   ```
   solve(a == L,rho_tilde, solver_parameters={"linear_solver": "cg", "preconditioner": "hypre_amg"})
   ```

   The uses conjugate gradient with an algebraic multigrid preconditioner (it can sometimes be necessary to use “gmres” or “bicgstab” with algebraic preconditioners instead of “cg”).

6. Make solutions using “petsc” (the ‘PETSc built in LU solver’, likely also the default solver) and “cg” with “hypre_amg” preconditioner and compare the solution times for different mesh sizes. You can measure the solution time using the supplied TicToc class as

   ```
   from ticToc import TicToc
   timer=TicToc(True)
   timer.tic("Direct Solver")
   ```

```
solve(a == L,rho_tilde,solver_parameters={"linear_solver": "petsc"})
timer.toc("Direct Solver")
```

"Direct Solver" is a label used when measuring and printing the elapsed time. Print also the number of degrees of freedom with

```
print("DOFs in linear problem: ",Vc.dim())
```

Try to make a graph of the solution time as a function of the number of degrees of freedom for the two types of solvers.

7. $\tilde{\rho}$ is generally a smoothed version of the input ρ. To make a sharp version of $\tilde{\rho}$, it can be passed through a "smoothed Heaviside function", implemented as

```
def tanh(x):
   return (exp(2*x)-1)/(exp(2*x)+1)
def Heavi(beta,eta,x,th=tanh):
   return (th(beta*eta)+th(beta*(x-eta)))/(th(beta*eta)+th(beta*(1-eta)))
```

To be able to evaluate the Heavi function for numpy vectors, one can use the "th" argument with numpy's version of tanh, e.g.

```
Heavi(1,0.5,x,th=np.tanh)
```

Make 1D plots of the smoothed Heaviside function for $x \in [0,1]$ with different values of β and η. numpy and pyplot are convenient for this, e.g., x=np.linspace(0,1,100) etc.
Compare to the Heaviside step function https://en.wikipedia.org/wiki/Heaviside_step_function .

8. The Heavi function can be used in FEniCS as

```
rho_bar=project(Heavi(32,0.4,rho_tilde),Vd)
```

or using the iterative solver

```
rho_bar=project(Heavi(32,0.4,rho_tilde),Vd,solver_type="cg",preconditioner_type="hypre_amg")
```

Without the th argument, Heavi uses the 'default' exp function inside tanh, which is defined by FEniCS to work with FEniCS's "Unified Form Language" (UFL) expressions.

Plot rho_bar and investigate what happens when the parameters β and η are changed.

9. Use this "Helmholtz filtering and Heaviside projection" to come up with a simple calculation that can produce a plot that shows the edges of the geometry (a the disc).